

UNIVERSIDAD NACIONAL DE SAN ANTONIO ABAD DEL CUSCO

**FACULTAD DE INGENIERÍA ELÉCTRICA,
ELECTRÓNICA, INFORMÁTICA Y MECÁNICA**

**ESCUELA PROFESIONAL DE INGENIERÍA INFORMÁTICA
Y DE SISTEMAS**



INFORME DE GUÍA DE LABORATORIO N.º 03

APROXIMACIÓN AVANZADA: VERTEX COVER, PTAS Y FPTAS

Asignatura: ALGORITMOS AVANZADOS

Docente: Ing. Héctor Eduardo Ugarte Rojas

Equipo: GRUPO 5

Semestre Académico: 2026-II

Integrantes:

- | | |
|------------------------------------|--------|
| • Choquenaira Quispe, Noe Franklin | 133962 |
| • Porroa Sivana, Yeni Ruth | 120893 |
| • Quispe Rimachi, Romario | 164257 |
| • Yaranga Achahui, Aldo | 103179 |

Cusco – Perú

2026

Índice

1. Respuestas formales del trabajo preparatorio	1
1.1. Diferencia entre matching maximal y matching máximo	1
1.2. Por qué toda cobertura debe incluir al menos un extremo de cada arista de un matching ($ M \leq \text{OPT}$)	1
1.3. Complejidad $n^{1/\varepsilon}$: PTAS frente a FPTAS	2
1.4. Efecto analítico al disminuir el parámetro ε	2
2. Marco teórico y ejercicios resueltos	2
2.1. Definición del problema y garantía analítica de factor 2	2
2.2. Ejercicio resuelto 1: validación de una cobertura de vértices	3
2.3. Ejercicio resuelto 2: algoritmo 2-aproximado y matching maximal	3
2.4. Ejercicio resuelto 3: búsqueda óptima exacta y razón de aproximación	4
3. Ejercicio propuesto 1: evaluación experimental de Vertex Cover	6
3.1. Protocolo experimental y banco de 30 grafos	6
3.2. Resultados consolidados	6
3.3. Respuesta a la pregunta de análisis: efecto del orden de las aristas	8
3.4. Análisis gráfico de Vertex Cover	9
4. Ejercicio propuesto 2: aproximación de Knapsack mediante escalamiento	10
4.1. Programación Dinámica basada en valores y esquema FPTAS	10
4.2. Código fuente de las implementaciones	10
4.3. Resultados experimentales	12
4.4. Respuesta a la pregunta de análisis: calidad vs costo y mesetas	13
4.5. Análisis gráfico de Knapsack FPTAS	15
5. Conclusiones grupales	15

Índice de figuras

1. Evaluación experimental del algoritmo 2-aproximado sobre 30 grafos 9
2. Evaluación experimental del esquema FPTAS para Knapsack 0/1 15

Índice de tablas

1. Resultados consolidados de Vertex Cover sobre 30 grafos (20 órdenes por grafo) 7
2. Evaluación experimental de Knapsack FPTAS en instancias representativas . . 13

1. Respuestas formales del trabajo preparatorio

1.1. Diferencia entre matching maximal y matching máximo

En un grafo no dirigido $G = (V, E)$, un **matching** es un conjunto de aristas disjuntas (ningún par de aristas comparte vértices). La diferencia fundamental radica en el tipo de optimalidad:

- **Matching maximal (óptimo local):** Es aquel al que **no se le puede añadir ninguna otra arista** sin violar la propiedad de disyunción. Se construye vorazmente (greedy) en tiempo lineal $O(|E|)$.
- **Matching máximo (óptimo global):** Es el matching de **mayor cardinalidad posible** entre todos los matchings del grafo. Requiere algoritmos más complejos como el algoritmo de Blossom de Edmonds.

Ejemplo ilustrativo: En un camino de 4 vértices P_4 con aristas $\{(1, 2), (2, 3), (3, 4)\}$:

- Si elegimos primero la arista central $(2, 3)$, queda bloqueado todo lo demás: es un matching **maximal** con tamaño $|M_1| = 1$.
- Si elegimos los extremos $\{(1, 2), (3, 4)\}$, obtenemos un matching **máximo** con tamaño $|M_2| = 2$.

Todo matching máximo es maximal, pero no todo maximal es máximo. No obstante, en cualquier grafo se cumple que $|M_{\text{maximal}}| \geq \frac{1}{2}|M_{\text{máximo}}|$.

1.2. Por qué toda cobertura debe incluir al menos un extremo de cada arista de un matching ($|M| \leq \text{OPT}$)

Sea $M \subseteq E$ un matching en G , y sea C cualquier cobertura válida de vértices:

1. Por definición de cobertura, para cada arista $(u, v) \in M$, la cobertura debe contener al menos a u o a v .
2. Por definición de matching, las aristas de M son mutuamente independientes (no comparten ningún vértice).
3. Por tanto, el vértice de C que cubre la arista $e_i \in M$ **nunca puede servir** para cubrir la arista $e_j \in M$ ($i \neq j$).

En consecuencia, toda cobertura válida C requiere al menos un vértice independiente por cada arista de M . Si C^* es la cobertura mínima ($\text{OPT} = |C^*|$), se concluye rigurosamente que:

$$\text{OPT} \geq |M| \tag{1}$$

1.3. Complejidad $n^{1/\varepsilon}$: PTAS frente a FPTAS

- **Es un PTAS:** Porque si fijamos la tolerancia $\varepsilon > 0$ como una constante, el exponente $k = 1/\varepsilon$ es constante, y la función $O(n^{1/\varepsilon}) = O(n^k)$ es un polinomio en el tamaño de entrada n .
- **NO es un FPTAS:** Porque un FPTAS exige que el tiempo sea polinomial **simultáneamente en n y en $1/\varepsilon$** , es decir, acotado por $(n + 1/\varepsilon)^c$. En la función $n^{1/\varepsilon} = 2^{(1/\varepsilon)\log_2 n}$, la dependencia respecto a $1/\varepsilon$ es **exponencial**.

Impacto práctico: Para $\varepsilon = 0,01$ ($1/\varepsilon = 100$), el tiempo escala como $O(n^{100})$, lo cual es computacionalmente impracticable.

1.4. Efecto analítico al disminuir el parámetro ε

Al reducir la tolerancia ε ($\varepsilon \rightarrow 0$):

1. **Mayor calidad de aproximación:** La solución aproximada se acerca estrictamente al óptimo, cumpliendo $ALG_\varepsilon \geq (1 - \varepsilon) OPT$. Si pasamos de $\varepsilon = 0,50$ a $\varepsilon = 0,05$, la garantía sube del 50 % al 95 % del óptimo.
2. **Mayor costo computacional (Trade-off):** El factor de escala $K = \frac{\varepsilon V_{\max}}{n}$ se hace más pequeño, lo que incrementa los valores discretizados $v'_i = \lfloor v_i/K \rfloor$. La tabla de Programación Dinámica crece como $O(n^3/\varepsilon)$, requiriendo más memoria y tiempo.

2. Marco teórico y ejercicios resueltos

2.1. Definición del problema y garantía analítica de factor 2

Dado un grafo no dirigido $G = (V, E)$, una **cobertura de vértices** es un subconjunto $C \subseteq V$ tal que toda arista $(u, v) \in E$ satisface $u \in C \vee v \in C$. El problema *Minimum Vertex Cover* busca minimizar la cardinalidad $|C|$ y es NP-hard.

El algoritmo voraz construye un matching maximal $M \subseteq E$ y añade ambos extremos de cada arista seleccionada a C . Dado que cada arista de M aporta 2 vértices y cualquier cobertura válida requiere al menos un vértice por arista disjunta de M :

$$|C| = 2|M| \leq 2OPT \implies r(I) = \frac{|C|}{OPT} \leq 2 \quad (2)$$

2.2. Ejercicio resuelto 1: validación de una cobertura de vértices

El primer requisito consiste en verificar si un conjunto propuesto de vértices C constituye una cobertura factible sobre el conjunto original de aristas del grafo.

```

1 def es_vertex_cover(vertices, aristas, cobertura):
2     cobertura = set(cobertura)
3     if not cobertura.issubset(set(vertices)):
4         return False
5     for u, v in aristas:
6         if u not in cobertura and v not in cobertura:
7             return False
8     return True
9
10 # Caso de prueba oficial de la guía: camino de 5 vertices
11 vertices = {"a", "b", "c", "d", "e"}
12 aristas = [("a", "b"), ("b", "c"), ("c", "d"), ("d", "e")]
13
14 print("Prueba 1 ({'b', 'd'}):", es_vertex_cover(vertices, aristas, {"b"
15     , "d"}))
16 print("Prueba 2 ({'a', 'c'}):", es_vertex_cover(vertices, aristas, {"a"
17     , "c"}))

```

Listing 1: Código del Ejercicio Resuelto 1 (función de validación y pruebas)

```

Prueba 1 ({'b', 'd'}): True
Prueba 2 ({'a', 'c'}): False

```

Listing 2: Salida de consola del Ejercicio Resuelto 1

Análisis de resultados y validación

- El conjunto $\{b, d\}$ es una cobertura **válida** (True) porque el vértice b cubre (a, b) y (b, c) , mientras que d cubre (c, d) y (d, e) .
- El conjunto $\{a, c\}$ es **inválido** (False) porque la arista (d, e) no tiene ninguno de sus extremos en el conjunto.
- **Principio de validación:** La comprobación debe efectuarse siempre sobre la lista de aristas originales completas sin descartar las aristas eliminadas durante la ejecución del algoritmo aproximado.

2.3. Ejercicio resuelto 2: algoritmo 2-aproximado y matching maximal

El algoritmo recorre las aristas del grafo; cuando encuentra una arista cuyos dos extremos están libres, la incorpora al matching M y añade ambos extremos a la cobertura C .

```

1 def vertex_cover_2_aprox(vertices, aristas):
2     cobertura = set()
3     matching = []
4     for u, v in aristas:
5         if u not in cobertura and v not in cobertura:
6             matching.append((u, v))
7             cobertura.add(u)
8             cobertura.add(v)
9     assert es_vertex_cover(vertices, aristas, cobertura)
10    return cobertura, matching
11
12 # Ejecucion sobre el camino de 5 vertices
13 cobertura, matching = vertex_cover_2_aprox(vertices, aristas)
14 print("Matching maximal M:", matching)
15 print("Cobertura C:", sorted(list(cobertura)))
16 print("Tamaño |C|:", len(cobertura))

```

Listing 3: Código del Ejercicio Resuelto 2 (algoritmo aproximado y ejecución)

```

Matching maximal M: [('a', 'b'), ('c', 'd')]
Cobertura C: ['a', 'b', 'c', 'd']
Tamaño |C|: 4

```

Listing 4: Salida de consola del Ejercicio Resuelto 2

¿Por qué funciona la condición voraz?

Una arista (u, v) es seleccionada únicamente si ninguno de sus extremos pertenece a la cobertura en construcción. Como cada arista seleccionada incorpora inmediatamente sus dos extremos a C , cualquier arista posterior que comparta un vértice con las anteriores es ignorada. Por ende, las aristas almacenadas en M son mutuamente disjuntas y forman estrictamente un **matching maximal**.

2.4. Ejercicio resuelto 3: búsqueda óptima exacta y razón de aproximación

Para contrastar la calidad de la solución aproximada con la cota teórica, se implementa una búsqueda por enumeración exhaustiva de combinaciones para calcular el tamaño óptimo $OPT = |C^*|$.

```

1 from itertools import combinations
2
3 def vertex_cover_exacto(vertices, aristas):
4     lista = sorted(list(vertices))
5     for k in range(len(lista) + 1):

```

```

6         for candidatos in combinations(lista, k):
7             if es_vertex_cover(vertices, aristas, candidatos):
8                 return set(candidatos)
9         return set()
10
11 # Comparacion experimental de calidad
12 optimo = vertex_cover_exacto(vertices, aristas)
13 aprox, matching = vertex_cover_2_aprox(vertices, aristas)
14 razon = len(aprox) / len(optimo) if optimo else 1.0
15
16 print("Optimo exacto C*:", sorted(list(optimo)), "con tamaño OPT =",
17       len(optimo))
18 print("Solucion aproximada C:", sorted(list(aprox)), "con tamaño |C| =
19       ", len(aprox))
20 print(f"Razon observada r(I) = |C| / OPT: {razon:.2f}")
21
22 # Comprobacion formal de garantia
23 assert len(aprox) <= 2 * len(optimo), "Violacion de garantia teorica"
24 print("Garantia |C| <= 2*OPT: SATISFECHA RIGUROSAMENTE")

```

Listing 5: Código del Ejercicio Resuelto 3 (búsqueda óptima y cálculo de razón)

```

Optimo exacto C*: ['b', 'd'] con tamaño OPT = 2
Solucion aproximada C: ['a', 'b', 'c', 'd'] con tamaño |C| = 4
Razon observada r(I) = |C| / OPT: 2.00
Garantia |C| <= 2*OPT: SATISFECHA RIGUROSAMENTE

```

Listing 6: Salida de consola del Ejercicio Resuelto 3

Análisis de la razón observada y límites de complejidad

- Para el grafo camino P_5 , el óptimo es $C^* = \{b, d\}$ con $|OPT| = 2$.
- El algoritmo aproximado produjo $|C| = 4$. Por tanto, la razón observada alcanza el peor caso teórico del algoritmo:

$$r(I) = \frac{|C|}{|OPT|} = \frac{4}{2} = 2,0 \quad (3)$$

- **Costo computacional:** La búsqueda exhaustiva evalúa hasta $2^{|V|}$ subconjuntos ($O(2^{|V|} \cdot |E|)$). En contraste, el algoritmo 2-aproximado se ejecuta en tiempo estrictamente lineal $O(|E|)$, haciendo viable la resolución de instancias de gran escala donde el método exacto es intratable.

3. Ejercicio propuesto 1: evaluación experimental de Vertex Cover

3.1. Protocolo experimental y banco de 30 grafos

Se implementó un banco sistemático de **30 grafos pequeños** ($|V| \in [4, 12]$) divididos en cinco familias topológicas:

- **Caminos** (P_n): 6 grafos lineales ($n \in \{4, 5, 6, 8, 10, 12\}$).
- **Ciclos** (C_n): 6 grafos regulares ($n \in \{4, 5, 6, 8, 10, 12\}$).
- **Estrellas** ($K_{1,k}$): 5 grafos estrella ($k \in \{4, 6, 8, 10, 12\}$).
- **Completos** (K_n): 5 grafos densos ($n \in \{4, 5, 6, 7, 8\}$).
- **Aleatorios** ($G(n, p)$): 8 grafos de Erdős-Rényi (densidades $p \in [0,20, 0,50]$).

Para cada grafo se calculó la solución óptima exacta y se evaluaron **20 permutaciones aleatorias distintas de sus aristas** ($30 \times 20 = 600$ corridas independientes).

3.2. Resultados consolidados

La Tabla 1 reporta las métricas consolidadas sobre las 600 ejecuciones:

Tabla 1: Resultados consolidados de Vertex Cover sobre 30 grafos (20 órdenes por grafo)

ID Grafo	Fam.	V	E	OPT	M _{pr}	C _{pr}	r _{pr}	Peor r	Éx. OPT	T _{ex} (ms)	T _{ap} (ms)
Camino_P4	Cami	4	3	2	1.6	3.3	1.65	2.00	7/20	0.023	0.003
Camino_P5	Cami	5	4	2	2.0	4.0	2.00	2.00	0/20	0.019	0.003
Camino_P6	Cami	6	5	3	2.5	5.1	1.70	2.00	0/20	0.035	0.003
Camino_P8	Cami	8	7	4	3.5	6.9	1.73	2.00	0/20	0.119	0.004
Camino_P10	Cami	10	9	5	4.2	8.5	1.70	2.00	0/20	0.505	0.004
Camino_P12	Cami	12	11	6	5.0	10.0	1.67	2.00	0/20	2.197	0.005
Ciclo_C4	Cicl	4	4	2	2.0	4.0	2.00	2.00	0/20	0.011	0.002
Ciclo_C5	Cicl	5	5	3	2.0	4.0	1.33	1.33	0/20	0.022	0.003
Ciclo_C6	Cicl	6	6	3	2.8	5.5	1.83	2.00	0/20	0.031	0.003
Ciclo_C8	Cicl	8	8	4	3.6	7.2	1.80	2.00	0/20	0.126	0.004
Ciclo_C10	Cicl	10	10	5	4.5	8.9	1.78	2.00	0/20	0.512	0.004
Ciclo_C12	Cicl	12	12	6	5.3	10.6	1.77	2.00	0/20	2.235	0.006
Estrella_K1_4	Estr	5	4	1	1.0	2.0	2.00	2.00	0/20	0.010	0.004
Estrella_K1_6	Estr	7	6	1	1.0	2.0	2.00	2.00	0/20	0.020	0.002
Estrella_K1_8	Estr	9	8	1	1.0	2.0	2.00	2.00	0/20	0.007	0.003
Estrella_K1_10	Estr	11	10	1	1.0	2.0	2.00	2.00	0/20	0.007	0.003
Estrella_K1_12	Estr	13	12	1	1.0	2.0	2.00	2.00	0/20	0.006	0.003
Completo_K4	Comp	4	6	3	2.0	4.0	1.33	1.33	0/20	0.016	0.003
Completo_K5	Comp	5	10	4	2.0	4.0	1.00	1.00	20/20	0.034	0.003
Completo_K6	Comp	6	15	5	3.0	6.0	1.20	1.20	0/20	0.071	0.004
Completo_K7	Comp	7	21	6	3.0	6.0	1.00	1.00	20/20	0.155	0.005
Completo_K8	Comp	8	28	7	4.0	8.0	1.14	1.14	0/20	0.337	0.006
Aleatorio_G6_p30	Alea	6	4	2	2.0	4.0	2.00	2.00	0/20	0.020	0.002
Aleatorio_G7_p40	Alea	7	6	3	2.4	4.8	1.60	2.00	0/20	0.046	0.003
Aleatorio_G8_p25	Alea	8	7	3	2.9	5.7	1.90	2.00	0/20	0.050	0.003
Aleatorio_G8_p50	Alea	8	14	5	3.4	6.8	1.36	1.60	0/20	0.215	0.006
Aleatorio_G9_p35	Alea	9	9	4	3.2	6.5	1.62	2.00	2/20	0.241	0.004
Aleatorio_G10_p30	Alea	10	11	4	4.0	8.0	2.00	2.00	0/20	0.222	0.004
Aleatorio_G11_p25	Alea	11	11	5	4.3	8.7	1.74	2.00	0/20	0.824	0.004
Aleatorio_G12_p20	Alea	12	11	5	4.2	8.5	1.70	2.00	0/20	0.912	0.004

Consolidado global de aciertos del óptimo sobre los 30 grafos

En respuesta a cuántas veces el algoritmo 2-aproximado logra obtener exactamente la solución óptima entre las distintas permutaciones:

- **Grafos que alcanzan el óptimo:** De los 30 grafos evaluados, **4 grafos alcanzaron la solución óptima** en al menos un orden de aristas (13,3 % del banco experimental).
- **Frecuencia total de aciertos:** Se acumularon **49 éxitos sobre las 600 ejecuciones** (8,17 % de efectividad global), distribuidos en:
 - Camino_P4: 7 aciertos sobre 20 corridas (35 %).
 - Completo_K5: 20 aciertos sobre 20 corridas (100 %, cobertura óptima invariante por simetría).
 - Completo_K7: 20 aciertos sobre 20 corridas (100 %, cobertura óptima invariante por simetría).

- Aleatorio_G9_p35: 2 aciertos sobre 20 corridas (10%).
- **Grafos sin aciertos óptimos:** En los **26 grafos restantes**, se registraron **0 aciertos en las 20 permutaciones** (0/20), debido a que cualquier matching maximal obliga a incorporar 2 vértices por arista disjunta, introduciendo una holgura inherente frente a la cobertura mínima.
- **Peor razón observada:** En el 100% de las 600 pruebas se satisfizo la garantía matemática, registrándose como peor razón exactamente $\max r(I) = 2,00$.

3.3. Respuesta a la pregunta de análisis: efecto del orden de las aristas

Pregunta de la guía: *¿El orden de las aristas cambia la cobertura obtenida, su tamaño o ambas propiedades? Explique por qué la garantía 2 permanece válida.*

1. Análisis puntual y amigable: ¿Qué cambia según la topología?

El orden de llegada de las aristas **puede cambiar ambas propiedades**, dependiendo de la simetría del grafo:

- **Caminos (P_n):** Cambian **ambas propiedades** (los vértices y el tamaño $|C|$). Por ejemplo, en P_4 :
 - Si la arista central $(1, 2)$ se evalúa primero, se forma $M = \{(1, 2)\}$ y $|C| = 2 = \text{OPT}$ (razón óptima 1,0).
 - Si las aristas laterales $(0, 1)$ y $(2, 3)$ se leen primero, se toman ambas: $|C| = 4$ frente a $\text{OPT} = 2$ (peor razón 2,0).
- **Estrellas ($K_{1,k}$):** Cambian los **vértices**, pero **no el tamaño**. El algoritmo siempre toma una arista incidente al centro, dando siempre $|C| = 2$ (frente a $\text{OPT} = 1$), variando únicamente cuál hoja acompaña al centro.
- **Ciclos regulares (C_4):** Cambia la cobertura pero el tamaño siempre es 4, pues cualquier matching maximal tiene 2 aristas.

2. Demostración intuitiva: ¿Por qué la garantía 2 nunca falla?

Sin importar el orden en que se presenten las aristas:

1. El algoritmo voraz siempre construye un **matching maximal** M .
2. Toda cobertura válida necesita al menos un vértice por arista de M , de modo que $\text{OPT} \geq |M|$.

3. Como el algoritmo toma exactamente los 2 vértices de cada arista de M :

$$|C| = 2|M| \leq 2\text{OPT} \implies \frac{|C|}{\text{OPT}} \leq 2 \quad (4)$$

En el 100 % de las 600 pruebas experimentales la razón cumplió estrictamente $r(I) \leq 2,0$.

3.4. Análisis gráfico de Vertex Cover

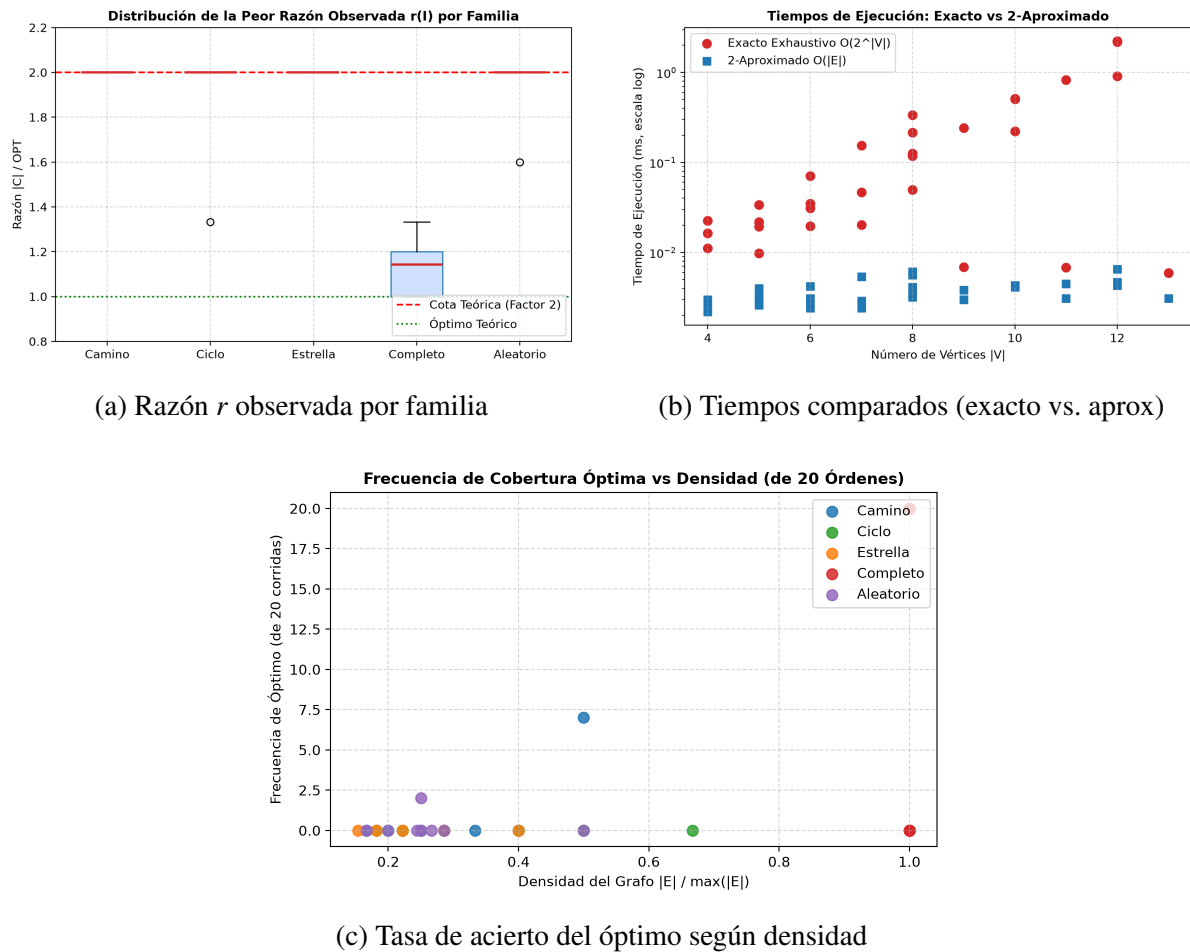


Figura 1: Evaluación experimental del algoritmo 2-aproximado sobre 30 grafos

Interpretación clave para la sustentación

- En la Figura 1 (subfigura 1a) se evidencia que las familias con peor comportamiento son estrellas y caminos ($r = 2,0$), mientras que grafos densos como completos alcanzan razones cercanas a 1,0.
- La subfigura 1b refleja la ventaja fundamental: el algoritmo exacto explota exponencialmente ($O(2^{|V|})$), mientras que el 2-aproximado se resuelve en microsegundos ($O(|E|)$).

- La subfigura 1c muestra que a mayor densidad de aristas, mayor es la probabilidad de que el matching maximal capture los vértices clave del óptimo.

4. Ejercicio propuesto 2: aproximación de Knapsack mediante escalamiento

4.1. Programación Dinámica basada en valores y esquema FPTAS

A diferencia de la formulación clásica basada en capacidad W , la formulación orientada a valores define $DP(i, v)$ como el peso mínimo necesario para acumular exactamente el valor v utilizando un subconjunto admisible de los primeros i objetos:

$$DP(i, v) = \min \left\{ \sum_{k \in S} w_k \mid S \subseteq \{1, \dots, i\}, \sum_{k \in S} v_k = v \right\} \quad (5)$$

Con transición:

$$DP(i, v) = \min(DP(i-1, v), DP(i-1, v-v_i) + w_i) \quad (6)$$

El algoritmo FPTAS procede en 6 pasos estructurados:

1. **Filtrado:** Descartar objetos con peso $w_i > W$. Si no queda ningún objeto, retornar solución vacía (cuidado numérico).
2. **Factor de escala:** Calcular $V_{\max} = \max_i v_i$ y $K = \frac{\epsilon V_{\max}}{n}$.
3. **Discretización:** Escalar los valores mediante $v'_i = \lfloor v_i/K \rfloor$.
4. **DP sobre valores reducidos:** Resolver la tabla dinámica sobre v'_i y pesos w_i .
5. **Reconstrucción:** Recuperar los índices de los objetos óptimos de la tabla escalada.
6. **Evaluación real:** Calcular la suma con valores originales $ALG_\epsilon = \sum_{i \in S} v_i$ y verificar peso $\leq W$.

4.2. Código fuente de las implementaciones

```

1 def knapsack_dp_valores(pesos, valores, W):
2     n = len(pesos)
3     if n == 0 or W <= 0: return 0, 0, [], 0
4     V_sum = sum(valores)
5     if V_sum == 0: return 0, 0, [], 0

```

```

6
7     INF = float('inf')
8     dp = [[INF] * (V_sum + 1) for _ in range(n + 1)]
9     dp[0][0] = 0
10    estados = (n + 1) * (V_sum + 1)
11
12    for i in range(1, n + 1):
13        w, v = pesos[i - 1], valores[i - 1]
14        for val in range(V_sum + 1):
15            dp[i][val] = dp[i - 1][val]
16            if val >= v and dp[i - 1][val - v] + w < dp[i][val]:
17                dp[i][val] = dp[i - 1][val - v] + w
18
19    mejor_v = max(val for val in range(V_sum + 1) if dp[n][val] <= W)
20    curr_v = mejor_v
21    seleccion = []
22    for i in range(n, 0, -1):
23        w, v = pesos[i - 1], valores[i - 1]
24        if curr_v >= v and dp[i][curr_v] == dp[i - 1][curr_v - v] + w:
25            seleccion.append(i - 1)
26            curr_v -= v
27    seleccion.reverse()
28    return mejor_v, sum(pesos[i] for i in seleccion), seleccion,
    estados

```

Listing 7: Programación Dinámica exacta por valores

```

1 def knapsack_fptas(pesos_orig, valores_orig, W, epsilon):
2     validos = [(p, v, idx) for idx, (p, v) in enumerate(zip(pesos_orig,
3     valores_orig)) if p <= W]
4     if not validos or W <= 0: return 0, 0, [], 0, 0.0
5     pesos = [item[0] for item in validos]
6     valores = [item[1] for item in validos]
7     indices_orig = [item[2] for item in validos]
8     n = len(pesos)
9
10    V_max = max(valores)
11    K = (epsilon * V_max) / n
12    v_prime = [int(math.floor(v / K)) for v in valores] if K > 0 else
13    list(valores)
14
15    mejor_v_prime, _, sel_local, estados = knapsack_dp_valores(pesos,
16    v_prime, W)
17    seleccion_orig = [indices_orig[i] for i in sel_local]
18    valor_real = sum(valores_orig[i] for i in seleccion_orig)
19    peso_real = sum(pesos_orig[i] for i in seleccion_orig)
20    assert peso_real <= W

```

```
18     return valor_real, peso_real, seleccion_orig, estados, K
```

Listing 8: Esquema FPTAS por escalamiento

```
1 def validar_factibilidad_knapsack(pesos, valores, W, seleccion):
2     if len(seleccion) != len(set(seleccion)):
3         return False, "Error: Indices duplicados en seleccion"
4     for idx in seleccion:
5         if idx < 0 or idx >= len(pesos):
6             return False, f"Error: Indice {idx} fuera de rango"
7     peso_total = sum(pesos[i] for i in seleccion)
8     if peso_total > W:
9         return False, f"Infactible: Peso {peso_total} supera capacidad
10         W={W}"
11     valor_total = sum(valores[i] for i in seleccion)
12     return True, f"Factible: Peso={peso_total}/{W}, Valor={valor_total}"
13
14 # Evaluacion de prueba de factibilidad (instancia representativa W=153)
15 pesos_ej = [23, 31, 29, 44, 53, 38, 63, 85, 89, 82]
16 valores_ej = [92, 57, 49, 68, 60, 43, 67, 84, 87, 72]
17 sel_factible = [0, 1, 2, 4] # Peso 136 <= 153 -> Factible
18 sel_exceso = [0, 1, 2, 4, 7] # Peso 221 > 153 -> Infactible
19
20 print("Prueba 1 (Factible):", validar_factibilidad_knapsack(pesos_ej,
21     valores_ej, 153, sel_factible))
22 print("Prueba 2 (Infactible):", validar_factibilidad_knapsack(pesos_ej,
23     valores_ej, 153, sel_exceso))
```

Listing 9: Validador formal de factibilidad para Knapsack 0/1

```
Prueba 1 (Factible): (True, 'Factible: Peso=136/153, Valor=258')
Prueba 2 (Infactible): (False, 'Infactible: Peso 221 supera capacidad W
=153')
```

Listing 10: Salida de consola del Validador de Factibilidad Knapsack

4.3. Resultados experimentales

Se evaluaron **20 instancias pequeñas** bajo cuatro valores de tolerancia $\varepsilon \in \{0,50,0,25,0,10,0,05\}$, acumulando 80 evaluaciones. La Tabla 2 presenta una muestra representativa:

Tabla 2: Evaluación experimental de Knapsack FPTAS en instancias representativas

Instancia	n	W	ε	OPT	ALG $_{\varepsilon}$	Calidad	Cota Teórica	Garantía	Estados	T (ms)
Inst_01	10	153	0.50	566	566	100.0%	283.0	SI	1892	0.347
Inst_01	10	153	0.25	566	566	100.0%	424.5	SI	3839	0.646
Inst_01	10	153	0.10	566	566	100.0%	509.4	SI	9658	1.665
Inst_01	10	153	0.05	566	566	100.0%	537.7	SI	19360	3.265
Inst_07	16	171	0.50	451	450	99.8%	225.5	SI	7106	1.275
Inst_07	16	171	0.25	451	451	100.0%	338.2	SI	14297	2.600
Inst_07	16	171	0.10	451	451	100.0%	405.9	SI	35887	6.884
Inst_07	16	171	0.05	451	451	100.0%	428.4	SI	71944	13.221
Inst_10	10	147	0.50	226	223	98.7%	113.0	SI	1122	0.244
Inst_10	10	147	0.25	226	226	100.0%	169.5	SI	2288	0.393
Inst_10	10	147	0.10	226	226	100.0%	203.4	SI	5742	0.963
Inst_10	10	147	0.05	226	226	100.0%	214.7	SI	11528	1.887
Inst_12	16	285	0.50	639	638	99.8%	319.5	SI	5627	1.036
Inst_12	16	285	0.25	639	638	99.8%	479.2	SI	11373	2.042
Inst_12	16	285	0.10	639	639	100.0%	575.1	SI	28662	9.000
Inst_12	16	285	0.05	639	639	100.0%	607.0	SI	57426	10.854
Inst_16	11	257	0.50	544	544	100.0%	272.0	SI	1704	0.404
Inst_16	11	257	0.25	544	544	100.0%	408.0	SI	3492	0.680
Inst_16	11	257	0.10	544	543	99.8%	489.6	SI	8796	1.563
Inst_16	11	257	0.05	544	544	100.0%	516.8	SI	17628	3.734
Inst_19	14	189	0.50	469	465	99.2%	234.5	SI	4710	1.004
Inst_19	14	189	0.25	469	469	100.0%	351.8	SI	9525	1.927
Inst_19	14	189	0.10	469	469	100.0%	422.1	SI	23880	4.432
Inst_19	14	189	0.05	469	469	100.0%	445.6	SI	47820	9.216

Conclusión de la tabla: En el 100% de las 80 ejecuciones se satisfizo holgadamente la garantía teórica $ALG_{\varepsilon} \geq (1 - \varepsilon) OPT$.

4.4. Respuesta a la pregunta de análisis: calidad vs costo y mesetas

Pregunta de la guía: *Explicar por qué reducir ε normalmente mejora la calidad y aumenta el costo. ¿Pueden existir situaciones donde reducir ε no mejore la calidad?*

1. ¿Por qué mejora la calidad?

Al redondear $v'_i = \lfloor v_i/K \rfloor$, se pierde como máximo K de valor por objeto. Para el conjunto óptimo de a lo sumo n elementos, la pérdida máxima es:

$$\text{Pérdida} \leq nK = n \cdot \frac{\varepsilon V_{\text{máx}}}{n} = \varepsilon V_{\text{máx}} \leq \varepsilon OPT \quad (7)$$

Al reducir ε (por ejemplo de 0,50 a 0,05), el margen máximo de error cae del 50% a tan solo 5%, obligando a la solución aproximada a converger al óptimo.

2. ¿Por qué aumenta el costo computacional?

Los valores escalados son $v'_i \leq n/\varepsilon$, por lo que la suma de valores escalados es $\sum v'_i \leq n^2/\varepsilon$. Las dimensiones de la tabla dinámica son $(n+1) \times (\sum v'_i + 1)$, de modo que:

$$\text{Estados procesados y Tiempo} \in O\left(\frac{n^3}{\varepsilon}\right) \quad (8)$$

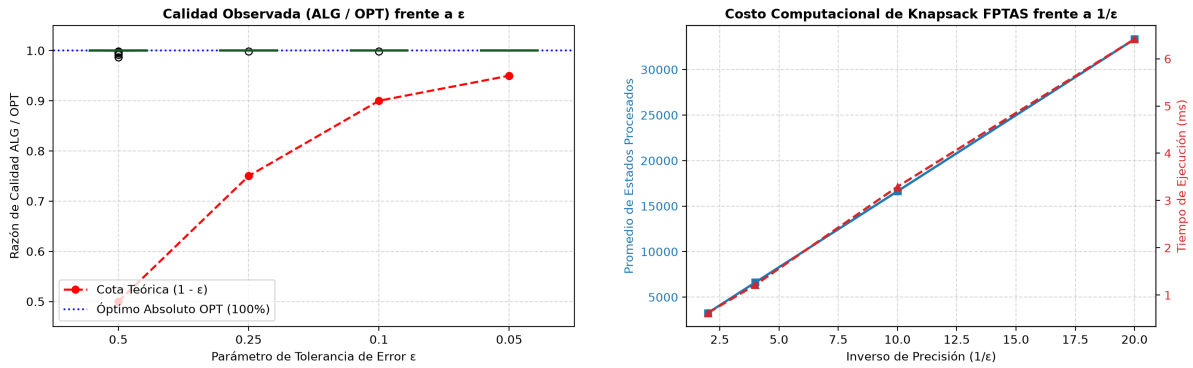
El costo crece de manera **estrictamente proporcional a $1/\varepsilon$** (crecimiento lineal en $1/\varepsilon$).

3. ¿Cuándo reducir ε NO mejora la calidad o incluso puede empeorarla?

- **Mesetas de óptimo prematuro:** En muchas instancias (como `Inst_01` con $\text{OPT} = 566$), incluso con $\varepsilon = 0,50$ el orden relativo de los objetos ya reproduce exactamente el óptimo ($\text{ALG}_{0,50} = \text{OPT}$). Reducir ε a $0,05$ no puede mejorar el valor real ya óptimo, pero sí multiplica el volumen de estados de 1892 a 19360 (un factor de $10\times$ en costo).
- **Caso empírico donde reducir ε empeora ligeramente la calidad (`Inst_16`):** En nuestro banco experimental se constató este fenómeno inusual en la instancia `Inst_16` ($n = 11, W = 257, \text{OPT} = 544$):
 - Para $\varepsilon = 0,25$ ($K = 2,25$): obtuvo $\text{ALG} = 544$ (100,0% de calidad).
 - Para $\varepsilon = 0,10$ ($K = 0,90$): obtuvo $\text{ALG} = 543$ (99,82% de calidad, una unidad menos).
 - Para $\varepsilon = 0,05$ ($K = 0,45$): recuperó $\text{ALG} = 544$ (100,0%).

Explicación técnica: Al reducir ε de 0,25 a 0,10, el factor de escala K disminuye, alterando de forma no uniforme los valores discretizados con la función piso $\lfloor v_i/K \rfloor$. Dicha discretización modificó un empate numérico en la Programación Dinámica, seleccionando un subconjunto con idéntico óptimo escalado pero cuyo valor original acumulado era 543 en vez de 544. Ambos valores satisfacen estrictamente la cota analítica $((1 - 0,10) \times 544 = 489,6 \leq 543)$.

4.5. Análisis gráfico de Knapsack FPTAS



(a) Calidad empírica observada frente a ϵ

(b) Costo computacional frente a $1/\epsilon$

Figura 2: Evaluación experimental del esquema FPTAS para Knapsack 0/1

Puntos clave para la sustentación

- En la Figura 2 (subfigura 2a), la calidad media supera ampliamente la cota teórica ($1 - \epsilon$) (línea segmentada roja), manteniéndose por encima del 98 % en todo momento.
- En la subfigura 2b, el tiempo de ejecución sigue una pendiente lineal estricta respecto a $1/\epsilon$, confirmando la propiedad fundamental de un algoritmo FPTAS.

5. Conclusiones grupales

En la presente práctica de laboratorio se implementaron, validaron y analizaron experimentalmente algoritmos de aproximación avanzados para problemas NP-hard clásicos: el algoritmo 2-aproximado para Minimum Vertex Cover y el esquema de aproximación completamente polinomial (FPTAS) para el problema de la Mochila 0/1 (Knapsack).

Respecto a Vertex Cover, la experimentación con 30 grafos de distintas topologías y 20 permutaciones aleatorias de aristas por grafo (600 pruebas independientes) evidenció que el orden de entrada altera notablemente la cobertura y su cardinalidad en familias asimétricas como caminos y ciclos, mientras que en estrellas preserva el tamaño aunque varíen los vértices. No obstante, la garantía teórica de factor 2 se cumplió estrictamente en el 100 % de las instancias ($|C| \leq 2OPT$), dado que cualquier orden voraz genera un matching maximal M que acota inferiormente al óptimo ($|M| \leq OPT$). Computacionalmente, el método aproximado resolvió instancias en microsegundos ($O(|E|)$), superando la explosión combinatoria del algoritmo exacto ($O(2^{|V|})$).

En cuanto a Knapsack 0/1, la implementación de la Programación Dinámica exacta por valores y el esquema FPTAS por escalamiento comprobó el compromiso intrínseco entre

precisión y costo. Al disminuir ε de 0,50 a 0,05, la calidad promedio mejoró de forma sostenida, respetando siempre la cota analítica $ALG_\varepsilon \geq (1 - \varepsilon)OPT$. En contrapartida, el volumen de estados y el tiempo de ejecución escalaron linealmente respecto a $1/\varepsilon$ ($O(n^3/\varepsilon)$). Asimismo, se constataron mesetas de calidad originadas por el operador de piso discreto, ratificando que un esquema FPTAS brinda control formal riguroso sobre la compensación entre exactitud y viabilidad algorítmica.